

Problem 1

Given a Binary Search Tree and a range **[low, high]**. Find all the numbers in the BST that lie in the given range.

Note: Element greater than or equal to root go to the right side.

Concept: Binary Search Tree

Example 1:

Input:

```
      17
     /  \
    4    18
   /  \
  2    9
l = 4, h = 24
```

Output:

4 9 17 18

Example 2:

Input:

```
      16
     /  \
    7    20
   /  \
  1    10
l = 13, h = 23
```

Output:

16 20

Company: Flipkart, Amazon, Microsoft

```
#include <stdio.h>
#include <stdlib.h>

// BST Node
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* newNode(int val) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = val;
    node->left = node->right = NULL;
    return node;
}

struct Node* insert(struct Node* root, int val) {
    if (root == NULL) return newNode(val);

    if (val < root->data)
        root->left = insert(root->left, val);
    else
        root->right = insert(root->right, val);

    return root;
}

void printInRange(struct Node* root, int low, int high) {
```

```

    if (root == NULL) return;

    if (root->data > low)
        printInRange(root->left, low, high);

    if (root->data >= low && root->data <= high)
        printf("%d ", root->data);

    if (root->data < high)
        printInRange(root->right, low, high);
}

int main() {
    struct Node* root = NULL;

    root = insert(root, 17);
    insert(root, 4);
    insert(root, 18);
    insert(root, 2);
    insert(root, 9);

    int low = 4, high = 24;
    printInRange(root, low, high); // Output: 4 9 17 18

    return 0;
}

```

Run

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:

0.0000 secs

Memory:

1.572 Mb

Your Output

4 9 17 18

Problem 2

Given a string **s**, composed of different combinations of '(', ')', '{', '}', '[', ']', Determine whether the Expression is **balanced** or not. An expression is balanced if:

1. Each opening bracket has a corresponding closing bracket of the same type.
2. Opening brackets must be closed in the correct order.

Concept: Stack

Examples :

Input: s = "[()]"

Output: true

Explanation: All the brackets are well-formed.

Input: s = "[()(){}]"

Output: true

Explanation: All the brackets are well-formed.

Input: s = "([)"

Output: false

Explanation: The expression is not balanced as there is a missing ')' at the end.

Input: s = "([)]"

Output: false

Explanation: The expression is not balanced as there is a closing ']' before the closing ']'.

Constraints:

$1 \leq s.size() \leq 106$

$s[i] \in \{ '(', ')', '{', '}', '[', ']' \}$

Company: Flipkart, Amazon, Microsoft, OYO, Oracle, Walmart, Adobe, Google

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

#define MAX 100000

char stack[MAX];
int top = -1;

void push(char c) {
    stack[++top] = c;
}
```

```

char pop() {
    return stack[top--];
}

bool isEmpty() {
    return top == -1;
}

bool isMatchingPair(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
}

bool isBalanced(char* s) {
    top = -1;

    for (int i = 0; i < strlen(s); i++) {
        char c = s[i];

        if (c == '(' || c == '{' || c == '[') {
            push(c);
        } else {
            if (isEmpty() || !isMatchingPair(pop(), c)) {
                return false;
            }
        }
    }

    return isEmpty();
}

int main() {
    char s[] = "{[()]}" ;

    if (isBalanced(s))
        printf("true\n");
    else
        printf("false\n");

    return 0;
}

```

Run

2520

Output

Status : Successfully executed

Time:

0.0000 secs

Memory:

1.492 Mb

Sample Input

2520

Your Output

true